

Estado del proyecto frente a los 11 puntos reclamados

Verificación línea por línea del código actual (frontend Angular + backend) contra el reclamo del cliente. Sin documentación de marketing, sin promesas de roadmap — solo lo que hoy corre.

1

Requerimiento completo y funcional

4

Parcial — existe pero no cumple lo pedido

6

No existe para el usuario final

REPOSITARIOS AUDITADOS

mantra-core-health
mantra-core-health-redesa-api

RAMA / ESTADO

dev (frontend), incl. working tree sin commitear

MÓDULOS BACKEND EN USO REAL

19 de 61 (~31%)

Metodología: lectura directa de código fuente. Nivel de madurez por punto: **completo** (lógica real conectada de punta a punta) · **parcial** (existe algo pero no cumple el pedido tal como se hizo) · **no existe** (sin pantalla y/o sin backend usable).

Resumen por punto

Los 11 puntos que el cliente reclamó como faltantes, en el mismo orden en que los planteó.

#	REQUERIMIENTO	ESTADO	EVIDENCIA CLAVE
1	Formularios por especialidad, configurables por doctor	No existe	Motor de formularios y plantillas por especialidad existen en el backend, sin CRUD completo ni ninguna pantalla en el frontend.
2	Generador de PDF	No existe	Sin librería de PDF en el frontend; sin generador en el backend. "PDF" solo aparece como valor de enum.
3	Catálogo maestro de servicios médicos	No existe	La tabla de catálogo existe en la base de datos pero no tiene endpoint ni pantalla — es un modelo huérfano.
4	Glosario de terminología médica, para cada profesional	Parcial	Buscador real y funcional, pero solo visible para administradores — no para el profesional de salud que lo necesita en consulta.
5	Presupuesto/cotización por paciente y procedimiento	No existe	Tablas de presupuesto en la base de datos sin lógica detrás; el menú de Facturación está apagado a propósito en el código.
6	Cita → atención → receta → ficha registrada	Completo	Flujo real de punta a punta: check-in de cita, apertura de encuentro vinculado, receta y diagnóstico, cierre de encuentro.
7	Histórico de procedimientos (cirujanos/odontólogos)	No existe	Backend con 37 entidades y 28 endpoints reales; cero pantallas o cliente en el frontend. Odontología no tiene ni un rastro.
8	Histórico de evoluciones como contexto de la cita	Parcial	El expediente sí carga al iniciar la atención, pero no hay forma de dar de alta una internación desde el frontend actual.
9	Red social profesional (carrera, hospitales, entidades)	Parcial	Lo más maduro de la auditoría — perfil, credenciales y muro de comunidad reales — pero sin ningún campo de historial laboral/hospitales.

#	REQUERIMIENTO	ESTADO	EVIDENCIA CLAVE
10	Registro de laboratorios e imagenología	No existe	Backend con 4 controllers y 20 endpoints reales; cero cliente y cero pantalla en el frontend.
11	Consultorios: dónde y cuándo atiende cada doctor	Parcial	Se puede reservar un horario real, pero no existe ningún campo ni pantalla que diga dónde atiende ese profesional.

Detalle por requerimiento

01 Formularios clínicos por especialidad, configurables por doctor

No existe

Compiten dos sistemas en el backend, y ninguno llega al usuario final. El módulo `forms` es un motor de formularios dinámicos completo (definiciones de campos, sets, asignaciones, instancias) en `redesa-api/src/modules/forms/controllers/*.ts`, pero el frontend no tiene ningún cliente para él — cero referencias a `FormsClient` o `/forms` en `src/app`.

Por otro lado, `chart.specialty_chart_templates` solo expone un endpoint para *asignar* una plantilla ya existente (`chart-templates.controller.ts:21-40`) — no hay forma de crear, listar ni leer el esquema de una plantilla por especialidad, ni siquiera desde el backend.

Conclusión: ningún doctor puede configurar un formulario clínico hoy, en ninguna capa.

02 Generador de PDF

No existe

El `package.json` del frontend no tiene `jspdf`, `pdfmake` ni `html2canvas`. El backend no tiene ningún generador de documentos — las únicas apariciones de "PDF" son valores de enum sin implementación detrás (`reporting.dto.ts:21-22`, `education.dto.ts:38-42`).

03 Catálogo maestro de servicios médicos

No existe

La entidad `billing.service_catalog` existe en la base de datos con código, nombre, precio por defecto e impuesto, pero no tiene ningún controller — es un modelo huérfano. Existe un endpoint para publicar un servicio de salud por práctica (`practices.controller.ts:175-188`), pero solo escribe: no hay un `GET` que liste el catálogo, y no maneja precio.

El frontend no tiene ninguna referencia a catálogo de servicios; el cliente de contabilidad solo expone plan de cuentas, diario y balance.

04 Glosario de terminología médica, accesible por cada profesional

Parcial — no cumple lo pedido

`TerminologyCatalog` (`terminology-catalog.ts`) es real: busca contra `GET /terminology/concepts` con paginación y estados de carga/error. El problema es a quién se le muestra.

Está registrado en `navigation.map.ts:189` restringido a `SECURITY_ADMIN` — no lo ve ningún médico. Además es un buscador técnico para resolver el UUID de un código de catálogo, no un glosario en lenguaje llano para consulta clínica. El propio comentario del código lo admite: "resolver un conceptId es una tarea de configuración, no de atención".

05 Presupuesto/cotización por paciente y procedimiento

No existe

Las tablas `billing.budgets` y `billing.budget_lines` existen en el backend, pero sin controller ni service que las use. En el frontend, la ruta de Facturación (`app.routes.ts:376`) está registrada en el menú pero marcada explícitamente `disabled: true` — el propio equipo la dejó apagada, no es un olvido.

06 Cita médica → inicio de atención → receta → ficha registrada

Completo

Es, junto con el punto 9, lo más sólido del sistema. Flujo verificado de punta a punta: `agenda.ts:694` hace check-in de la cita; desde ahí se abre el expediente y `patient-chart.ts:668-706` inicia el encuentro enlazando explícitamente la cita de origen como clave foránea real.

La receta se crea en `medication-block.ts:380` con firma e emisión como pasos separados; el diagnóstico en `diagnosis-block.ts:290`; y `patient-chart.ts:730-740` cierra el encuentro, lo que según el propio código dispara la facturación del lado del servidor. Todo lo escrito se vuelve a leer inmediatamente — no son formularios que "tragan" el dato.

Nota menor: existen métodos para reporte diagnóstico y verificación de interacciones farmacológicas en el cliente clínico, pero ninguna pantalla los invoca todavía — contrato listo, sin UI.

07 Histórico de procedimientos (cirujanos y odontólogos), equipos y notas

No existe

El backend tiene un módulo perioperatorio muy completo: 37 entidades y 28 endpoints reales en un solo controller (`periop.controller.ts:102-449`) — casos, pasos operativos, hallazgos, implantes, equipos de anestesia, estadías post-anestesia.

En el frontend no existe ni un archivo relacionado: sin cliente, sin pantalla. Para odontología específicamente no hay ni entidad, ni concepto, ni rastro en ningún lado del sistema.

08 Histórico de evoluciones (internación y no internación) como contexto de la cita

Parcial

Advertencia de nomenclatura: el módulo backend llamado `health_context` no es esto — es un módulo de epidemiología pública a nivel país, sin relación con evoluciones de pacientes.

Lo que sí resuelve el punto son los módulos `chart` y `clinical`: al abrir el expediente se cargan resumen e historial juntos (`patient-chart.ts:782-783`), y ese expediente es al que se llega desde la agenda — sí funciona como contexto real al iniciar la atención.

Limitación: no hay ningún método en el cliente clínico para crear un episodio de internación desde el frontend — solo se lee el campo si ya existe.

09 Red social profesional: carrera, hospitales, entidades médicas

Parcial — lo más maduro, con un hueco concreto

Es la parte mejor construida de todo lo auditado. `practitioner-profile.ts` llama a `GET /profiles/practitioners/me/summary` y muestra especialidades, formación, matrículas por jurisdicción con lógica real de vencimiento. La edición hace escrituras reales contra el backend, y el muro de comunidad (`CommunityClient`) funciona de verdad, con "artículos médicos" resueltos como posts con un hashtag específico — una solución pragmática, no un engaño.

El hueco: el cliente pidió explícitamente "hospitales y entidades médicas donde trabajó". El tipo de perfil solo tiene especialidades, credenciales (dónde se *formó*, no dónde trabajó), matrículas e idiomas — no existe ningún campo de historial de empleo/afiliación institucional, ni en frontend ni en backend.

Además, el backend de comunidad ya expone grupos, mensajería, encuestas y reseñas — relevante justo para "más información del profesional" — pero ninguno tiene cliente ni pantalla todavía.

10 Registro de laboratorios e imagenología

No existe

El backend tiene 4 controllers reales y 20 endpoints (laboratorio, imagenología, reportes, muestras) en `redesa-api/src/modules/diagnostics/`, más un módulo de equipamiento y sitios diagnósticos. En el frontend no hay ni cliente ni pantalla — cero coincidencias reales fuera de texto suelto en pantallas de diseño sin conectar.

11 Consultorios: dónde y con qué horario atiende cada doctor

Parcial — falta el "dónde"

La agenda de citas sí está genuinamente conectada — reservar un horario funciona de verdad. Pero el tipo de recurso agendable no tiene ningún campo de sitio, consultorio o dirección, y la pantalla de reserva no muestra ubicación en ningún punto.

El backend sí modela consultorios y sitios (`practice.sites.controller.ts`), pero el frontend no tiene ningún cliente para ese recurso. Hoy se puede saber *cuándo* atiende un profesional, pero no *dónde*.

NOTA TRANSVERSAL — EL ÁRBOL "REDSAT"

Un hallazgo que condiciona la lectura de varios puntos: `src/app/features/redsats/` (142 componentes) es en su totalidad interfaz sin lógica — ningún componente de ese árbol inyecta un cliente HTTP (verificado por búsqueda exhaustiva). Es una vista de diseño con apariencia funcional, no una implementación. Justo toca los puntos 4 y 11 (terminología y sucursales), por lo que se lo distingue explícitamente de las implementaciones reales encontradas en otros módulos.

Contexto adicional — módulo de tutoriales

No es parte de los 11 puntos reclamados, pero es trabajo reciente relevante para el estado general del producto.

El motor de tutoriales guiados (`src/app/core/tutorials/` , `tutorials-center.ts` , `tutorial-overlay`) es una funcionalidad autocontenida y genuinamente terminada: 34 definiciones de recorrido, overlay real que resalta elementos del DOM, progreso persistido en `localStorage` con degradación a memoria si no está disponible, y pruebas junto a cada pieza. No depende del backend y ya está integrado en el menú principal.

Cobertura general del backend

61	19	~31%
módulos backend en total	módulos con al menos una llamada real desde el frontend	del backend está efectivamente en uso

Entre los 42 módulos backend sin ningún consumo desde el frontend actual están, entre otros, exactamente los que sustentan la mayoría de los puntos reclamados: `billing` , `forms` , `procedures_perioperative` , `diagnostics` , `diagnostic_units` , `pharmacy` , `insurance` , `consent` , `document_store` y `reporting` , entre otros 33 módulos adicionales de plataforma e infraestructura.

Conteo por búsqueda estática de patrones (inyección de cliente, rutas invocadas, controllers declarados) — cota razonable, no una prueba exhaustiva en runtime.